



For the array `[10, 22, 9, 33, 21, 50, 41, 60]`, the length of the Longest Increasing Subsequence (LIS) is 1.

## Computer Algorithm : Assignment 02

2026. 5. 6. 16:59 · 카테고리 없음 수정 삭제 공개

There are actually two valid longest increasing subsequences for this specific array. Depending on how the algorithm breaks ties, it will return one of these:

1. `[10, 22, 33, 50, 60]`

### 목차



#0. Prompt



# Assignment 2 - 1 : Longest Increase Sequence (LIS)



Step 1: Prompt an AI to write Python code that finds the LIS for the array `[10, 22, 9, 33, 21, 50, 41, 60]` and prints the a...



Step2: AI often solves this by storing entire arrays for every state, which wastes memory. Analyze the space complexi...



# Assignment 2-2 :



Step 1



Step 2.



Step 3



Step 4



#Assignment 2-3 : All-Pair-Short est-Path Problem



- Course: Computer Algorithm
- StudentID: 2466044, 2272007
- Author: Sieun Lee, Yerin Kim
- Date: 2026-05-07
- Repository(public):<https://github.com/sihyes/2026-ComputerAlgorithm/tree/main/Assignment02>

### #0. Prompt

프롬프트는 과제에 나와있는 명령을 그대로 사용하였으며, 코딩에 뛰어난 Gemini 3.1 Pro 모델을 사용하여 응답을 생성했다.



프롬프트 복사

finds the LIS for the array [10, 22, 9, 33, 21, 50, 41, 60] and prints the actual elements of that sequence (not just the length)

## # Assignment 2 - 1 : Longest Increase Sequence (LIS)

# Hands-on assignment 2-1: Longest increasing subsequence

**Problem definition:** We discussed one possible implementation of LIS (Refer to previous slide26 ). The goal is to track and print the actual sequence, but you will use AI to achieve this optimality.

- Step1: Prompt an AI to write Python code that finds the LIS for the array [10, 22, 9, 33, 21, 50, 41, 60] and prints the actual elements of that sequence (not just the length). Document your prompt and the AI's initial output. (HINT: Need an additional variable to store the LIS)
- Step2: AI often solves this by storing entire arrays for every state, which wastes memory. Analyze the space complexity of the AI's initial code.
- Step3: If the AI's space complexity is inefficient, continue prompting the AI: "Optimize the space complexity. Do not store full arrays for each state; instead, use a 1D **prev** array to track the indices and reconstruct the sequence at the end." Document this interaction.
- Step4: Submit the final optimized code. Note that your implementation must return both the length of the LIS and the exact elements of the LIS. Briefly explain how the array successfully reconstructs the path.

EWHA,  
THE FUTURE

**problem definition :** We discussed one possible implementation of LIS (Refer to previous slide26 ). The goal is to track and print the actual sequence, but you will use AI to achieve this optimality.

Step 1: Prompt an AI to write Python code that finds the LIS for the array [10, 22, 9, 33, 21, 50, 41, 60] and prints the actual elements of that sequence (not just the length). Document your prompt and the AI's initial output. (HINT: Need an additional variable to store the LIS)

사용한 프롬프트는 다음과 같다.

```

1 finds the LIS for the array [10, 22, 9, 33, 21, 50, 41, 60] and prints the actual elements of that sequence (not just the length)

```

AI가 구현한 코드

```

def findLIS(arr):
    n = len(arr)
    dp = [1] * n
    prev = [-1] * n
    max_len = 1
    end_index = 0

    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
                prev[i] = j
        if dp[i] > max_len:
            max_len = dp[i]
            end_index = i

    # Reconstruct the LIS
    lis = []
    i = end_index
    while i != -1:
        lis.append(arr[i])
        i = prev[i]

    lis.reverse()
    return max_len, lis

```

```

1 def find_and_print_lis(arr):
2     n = len(arr)
3     if n == 0:
4         return []
5
6     # dp[i] stores the length of the LIS ending at index i
7     dp = [1] * n
8
9     # parent[i] stores the index of the previous element in the LIS
10    # We use this to reconstruct the actual path
11    parent = [-1] * n
12
13    # Build the dp array and track the parents
14    for i in range(1, n):
15        for j in range(i):
16            if arr[i] > arr[j] and dp[i] < dp[j] + 1:
17                dp[i] = dp[j] + 1
18                parent[i] = j
19

```

Step2: AI often solves this by storing entire arrays for every state, which wastes memory. Analyze the space complexity of the AI's initial code.

사용된 주요 자료구조는 다음과 같다. `dp[n]` , `parent[n]` , `lis` 이렇게 세 개의 추가적인 배열을 사용하고있다.

이때, `dp` , `parent` 의 경우, 길이가 모두  $n$ 이고, `lis` 의 경우 최대 길이가  $n$ 이 된다. `lis` 는 최종 결과를 저장하는 용도로 사용된다.

따라서 총 space 사용량은 각 배열의 크기와 같다.

`dp` :  $O(n)$

`parent` :  $O(n)$

`lis` :  $O(n)$  (In worst case)

따라서, 전체 Space Complexity는  $O(n)$  이다.

해당 코드에서는 every state를 저장하고 있지 않다.

- `dp[i]` = 길이만 저장 (int 1개)
- `parent[i]` = 이전 인덱스만 저장 (int 1개)

이렇게 저장하고있으므로, 경로 자체는 저장하지 않고 필요한 최소정보만 저장한뒤 마지막에 복원하는 방식으로 이루어지고 있다.

우려하던 전체 배열 저장 방식 예 : `dp[i] = [10, 22, 33]` 은 발생하지 않았다. (이 경우, 각 i마다 완전한 lis 배열을 저장하므로 총  $O(n^2)$ 의 space가 발생한다.)

따라서, 이미 잘 최적화된 코드이므로, step 3에서 이야기하는 prev array 기반 optimization 요구사항을 이미 만족하는 수준이다.

따라서 Step3, step 4는 생략한다.

Step3: If the AI's space complexity is inefficient, continue prompting the AI: "Optimize the space complexity. Do not store full arrays for each state; instead, use a 1D prev array to track the indices and reconstruct the sequence at the end." Document this interaction. → **skip**

• Step4: Submit the final optimized code. Note that your implementation must return both the length of the LIS and the exact elements of the LIS. Briefly explain how the array successfully reconstructs the path. → **skip**

## # Assignment 2-2 :

### Step 1

다음 사진과 같이 AI(Gemini Pro 버전)를 사용하여 코드를 생성하였다.



To send letter to my friend, Nate, it costs 140 cents. We have nine postage stamps and each costs, 1, 10, 21, 34, 70, 100, 350, 1225, 1500 cents. Implement a Dynamic Programming solution to find the minimum count of postage stamps for the 140-cent target. 파이썬 코드로 작성하고, 주석과 답변은 한국어로 해줘.



생각하는 과정 표시

친구 Nate에게 편지를 보내기 위한 최소 우표 개수를 구하는 문제는 대표적인 동적 계획법(Dynamic Programming) 알고리즘인 **동전 교환(Coin Change)** 문제와 동일합니다.

요청하신 대로 파이썬 코드로 구현하였으며, 코드 내 주석과 설명은 모두 한국어로 작성했습니다. 단순히 최소 개수뿐만 아니라, **어떤 우표들을 조합해야 하는지** 역추적하는 기능도 포함했습니다.

Python



```
def find_minimum_stamps(target, stamps):  
    # dp[i]는 금액 i를 만들기 위해 필요한 최소 우표 개수  
    # 초기값은 무한대(float('inf'))로 설정  
    dp = [float('inf')] * (target + 1)
```

생성한 전체 코드는 다음과 같다.

```
1 def find_minimum_stamps(target, stamps):
```



```

2 # dp[i]는 금액 i를 만들기 위해 필요한 최소 우표 개수
3 # 초기값은 무한대(float('inf'))로 설정
4 dp = [float('inf')] * (target + 1)
5
6 # 각 금액을 만들 때 마지막으로 사용한 우표의 액면가를 저장하는 배열 (역추적 용도)
7 used_stamp = [-1] * (target + 1)
8
9 # 0원을 만드는 데 필요한 우표는 0개
10 dp[0] = 0
11
12 # 1센트부터 목표 금액(target)까지 반복하며 최소 우표 개수 계산
13 for i in range(1, target + 1):
14     for stamp in stamps:
15         # 현재 금액(i)이 우표의 액면가보다 크거나 같을 때만 갱신 가능
16         if i >= stamp:
17             # 기존에 알려진 최소 개수보다, 현재 우표를 사용했을 때의 개수가 더 적다면 갱신
18             if dp[i - stamp] + 1 < dp[i]:
19                 dp[i] = dp[i - stamp] + 1

```

python

코드를 실행한 결과는 다음과 같다.

```

PS C:\Users\2272007\Desktop\컴알> c:;
\컴알\ha2_2_step1.py
목표 금액: 140센트
필요한 최소 우표 개수: 2개
사용된 우표 조합: [70, 70]

```

## Step 2.

위에서 생성한 코드에 DP 테이블을 모두 출력하는 코드를 추가하여 결과를 확인해보았다.

수정한 전체 코드는 다음과 같다.

```

1 def find_minimum_stamps(target, stamps):
2     # dp[i]는 금액 i를 만들기 위해 필요한 최소 우표 개수
3     # 초기값은 무한대(float('inf'))로 설정
4     dp = [float('inf')] * (target + 1)
5
6     # 각 금액을 만들 때 마지막으로 사용한 우표의 액면가를 저장하는 배열 (역추적 용도)
7     used_stamp = [-1] * (target + 1)
8
9     # 0원을 만드는 데 필요한 우표는 0개
10    dp[0] = 0
11
12    # 1센트부터 목표 금액(target)까지 반복하며 최소 우표 개수 계산

```

```

13     for i in range(1, target + 1):
14         for stamp in stamps:
15             # 현재 금액(i)이 우표의 액면가보다 크거나 같을 때만 갱신 가능
16             if i >= stamp:
17                 # 기존에 알려진 최소 개수보다, 현재 우표를 사용했을 때의 개수가 더 적다면 갱신
18                 if dp[i - stamp] + 1 < dp[i]:
19                     dp[i] = dp[i - stamp] + 1

```

코드를 실행한 결과는 다음과 같다. (DP table은 가장 마지막 10개의 항목이 보이도록 캡처했다.)

|           |    |                          |
|-----------|----|--------------------------|
| 128 cents | 6개 | [1, 1, 1, 21, 34, 70]    |
| 129 cents | 7개 | [1, 1, 1, 1, 21, 34, 70] |
| 130 cents | 4개 | [10, 10, 10, 100]        |
| 131 cents | 3개 | [10, 21, 100]            |
| 132 cents | 4개 | [1, 10, 21, 100]         |
| 133 cents | 4개 | [21, 21, 21, 70]         |
| 134 cents | 2개 | [34, 100]                |
| 135 cents | 3개 | [1, 34, 100]             |
| 136 cents | 4개 | [1, 1, 34, 100]          |
| 137 cents | 5개 | [1, 1, 1, 34, 100]       |
| 138 cents | 3개 | [34, 34, 70]             |
| 139 cents | 4개 | [1, 34, 34, 70]          |
| 140 cents | 2개 | [70, 70]                 |

목표 금액 : 140센트  
 필요한 최소 우표 개수 : 2개  
 사용된 우표 조합 : [70, 70]

DP table을 처음부터 살펴본 결과, Greedy 알고리즘과 DP의 결과가 처음으로 달라지는 숫자는 30이다.

Greedy 알고리즘에서는 30을 만들기 위해 먼저 21센트 우표를 사용하고, 남은 금액을 1센트 우표 9개로 채워서 총 10개의 우표를 사용한다. (local optimum)

그러나 DP 알고리즘에서는 10센트 우표 3장을 사용하여 greedy 알고리즘보다 훨씬 적은 수의 우표를 사용하는 global optimum을 찾았다.

이러한 현상이 발생하는 이유는, greedy 알고리즘은 가장 큰 값부터 순서대로 선택하기 때문에 더 작은 값들의 조합으로 이루어지는 경우를 고려하지 않는 반면, DP 알고리즘은 모든 경우의 수를 고려하며 누적해오는 방식으로 가장 우표를 적게 쓰는 조합을 찾기 때문이다.

|          |    |                              |
|----------|----|------------------------------|
| 20 cents | 2개 | [10, 10]                     |
| 21 cents | 1개 | [21]                         |
| 22 cents | 2개 | [1, 21]                      |
| 23 cents | 3개 | [1, 1, 21]                   |
| 24 cents | 4개 | [1, 1, 1, 21]                |
| 25 cents | 5개 | [1, 1, 1, 1, 21]             |
| 26 cents | 6개 | [1, 1, 1, 1, 1, 21]          |
| 27 cents | 7개 | [1, 1, 1, 1, 1, 1, 21]       |
| 28 cents | 8개 | [1, 1, 1, 1, 1, 1, 1, 21]    |
| 29 cents | 9개 | [1, 1, 1, 1, 1, 1, 1, 1, 21] |
| 30 cents | 3개 | [10, 10, 10]                 |
| 31 cents | 2개 | [10, 21]                     |

위 Step 2의 코드에서 사용 가능한 우표 조합을 [5,4,1]로 바꾸고, 목표 금액을 7센트로 바꾸어 코드를 실행한 결과는 다음과 같다.

```
PS C:\Users\2272007\Desktop\컴알>
\컴알\ha2_2_step3.py
1 cents | 1개 | [1]
2 cents | 2개 | [1, 1]
3 cents | 3개 | [1, 1, 1]
4 cents | 1개 | [4]
5 cents | 1개 | [5]
6 cents | 2개 | [5, 1]
7 cents | 3개 | [5, 1, 1]
목표 금액 : 7센트
필요한 최소 우표 개수 : 3개
사용된 우표 조합 : [5, 1, 1]
```

우표 조합과 목표 금액을 바꾼 경우에도 결과가 잘 나온 것을 확인할 수 있다.

Greedy 알고리즘으로 이 문제를 푼 경우, 가장 큰 금액인 5센트를 먼저 선택하고 1센트 두 개로 남은 2센트를 채우게 되므로 DP 알고리즘과 동일한 결과를 도출한다.

이번에는 목표 금액을 8센트로 바꾸어 코드를 실행해보았다.

```
PS C:\Users\2272007\Desktop\컴알>
\컴알\ha2_2_step3.py
1 cents | 1개 | [1]
2 cents | 2개 | [1, 1]
3 cents | 3개 | [1, 1, 1]
4 cents | 1개 | [4]
5 cents | 1개 | [5]
6 cents | 2개 | [5, 1]
7 cents | 3개 | [5, 1, 1]
8 cents | 2개 | [4, 4]
목표 금액 : 8센트
필요한 최소 우표 개수 : 2개
사용된 우표 조합 : [4, 4]
```

목표 금액을 바꾼 경우에도 가능한 최소 조합인 우표 2개 사용으로 결과가 잘 나온 것을 확인할 수 있다.

Greedy 알고리즘으로 이 문제를 푼 경우, 가장 큰 금액인 5센트를 먼저 선택하고 1센트 세 개로 남은 3센트를 채우게 되므로 총 3개의 우표를 사용한다.

이것은 DP 알고리즘의 결과인 우표 두 개보다 많으므로 greedy 알고리즘으로는 global optimum을 보장할 수 없음을 알 수 있다.

## Step 4

- Worst-case time complexity

목표 금액을  $V$ , 주어진 우표 종류의 개수를  $N$ 이라 할 때  $V$ 번 반복하는 외부 반복문과  $N$ 번 반복하는 내부 반복문을 돌면서 계산하므로 worst-case time complexity는  $\Theta(VN)$ 이다.

- Worst-case space complexity

각 cost별로 상태를 저장하고 역추적하기 위해 두 개의  $V+1$  크기의 배열 `dp`, `used_stamp`를 사용한다. 따라서 worst-case

space complexity는  $\Theta(V)$ 이다.

- DP가 Greedy 알고리즘이 실패하는 복잡한 문제에서도 성공하는 이유

우표 문제에서 큰 금액의 최적해는 작은 금액의 최적해에 우표를 하나씩 추가하는 방식으로 이루어진다. 즉, 큰 문제의 정답이 작은 subproblem들의 정답으로 이루어지는 optimal substructure이다.

Greedy 알고리즘은 지금 순간에서 가장 큰 값만 고르기 때문에 하위 단위의 더 효율적인 조합을 놓칠 수 있지만, DP는 한 번 계산한 작은 금액(overlapping subproblem)의 최적해를 저장해두고 이를 재사용하면서 더 높은 금액의 해를 찾아가며 모든 가능한 조합의 경우의 수를 누적하여 고려하므로 global optimum을 도출해 낼 수 있다

## #Assignment 2-3 : All-Pair-Short est-Path Problem

**Problem definition: Can we print all the vertices in the shortest path?**

**Step1: Change/add codes in Slide 18 to print out all the vertices in the shortest path from  $vs$  to  $vd$  (HINT:**

**Modify min function)**

슬라이드 18의 기존 코드에서 min() 함수 부분을 수정하고, 경로 추적을 위한 변수를 추가하여 최단 경로의 모든 정점을 출력하도록 변경하였다.

## Example 7: Floyd-Warshall's Algorithm for All-Pair-Shortest-Path Problem

```
INF = int (1e9 + 7)

def floyd_warshall(dist,length):
    for r in range(length):
        for p in range(length):
            for q in range(length):
                dist[p][q] = min(dist[p][q], dist[p][r] + dist[r][q])
    print_dist(dist,length)

def print_dist(dist,length):
    for p in range(length):
        temp=""
        for q in range(length):
            if(dist[p][q] == INF):
                temp+="INF "
            else:
                temp+=str(dist[p][q])+" "
        print(temp+" ")
```

```
# test code
W=[[0,4,11],
   [6,0,2],
   [3,INF,0]]
floyd_warshall(W,len(W[0]))
```

```
0 4 6
5 0 2
3 7 0
```

Time complexity:  $O(n^3)$   
Space complexity:  $O(n^2)$

EW/HA,  
THE FUTURE  
WE CREATE

18

기존 코드

다음과 같이 변경하였다.





```

1 # 거리가 갱신될 때 경로도 업데이트
2 if dist[p][q] > dist[p][r] + dist[r][q]:
3     dist[p][q] = dist[p][r] + dist[r][q]
4     next_node[p][q] = next_node[p][r]

```

HINT에 따라 min 함수를 if 조건문으로 변경하고, 경로 정보를 업데이트하는 로직을 추가했다.

이 구조는 p에서 q로 가는 최단 경로 상의 첫 번째 중간 노드를 효과적으로 기록한다.

거리가 짧아지는 순간  $\text{next\_v}[p][q] = \text{next\_v}[p][r]$ 을 통해 경로를 저장하도록 했다.

## Step 2:

다음과 같이 인풋을 넣어주었다.

## Step 3

: (v1, v3) 및 (v0, v2)의 최단 경로 정점 출력.

```

1 INF = int(1e9 + 7)
2
3 # Step 1: 경로를 기록하도록 플로이드-워셜 알고리즘 수정
4 def floyd_warshall_with_path(W, length):
5     # 최단 거리 배열 초기화
6     dist = [[W[i][j] for j in range(length)] for i in range(length)]
7
8     # 다음 방문할 정점을 저장할 배열 초기화
9     next_v = [[None] * length for _ in range(length)]
10    for i in range(length):
11        for j in range(length):
12            # 간선이 존재하면 도착점을 다음 방문 노드로 설정
13            if dist[i][j] != INF and i != j:
14                next_v[i][j] = j
15
16    # Floyd-Warshall 3중 루프
17    for r in range(length):
18        for p in range(length):
19            for q in range(length):

```

출력결과

```
$ python -u "d:\3-1\ComputerAlgorithm\Assignment02\d.py"
=== 플로이드-워셜 최단 경로 추적 결과 ===
(v1, v3) 의 최단 경로 정점들: v1 -> v2 -> v3
(v0, v2) 의 최단 경로 정점들: v0 -> v1 -> v2
```

### $v_1$ 에서 $v_3$ 로 가는 최단 경로 (1, 3)

- 주어진 W 행렬에 따르면  $v_1 \rightarrow v_3$ 로 가는 직접적인 간선은 없다 (INF).
- 다만,  $v_1 \rightarrow v_2$  (거리 2)를 거쳐  $v_2 \rightarrow v_3$  (거리 1)로 가면 총 거리가 3으로 최단 경로가 된다.
- 따라서 출력된 정점 리스트는  $v_1 \rightarrow v_2 \rightarrow v_3$  이다.

### $v_0$ 에서 $v_2$ 로 가는 최단 경로 (0, 2)

- $v_0 \rightarrow v_2$ 로 가는 직접 간선 역시 INF이다.
- 하지만  $v_0 \rightarrow v_1$  (거리 3)을 거쳐  $v_1 \rightarrow v_2$  (거리 2)로 가면 총 거리가 5로 최단 경로가 된다.
- 따라서 출력된 정점 리스트는  $v_0 \rightarrow v_1 \rightarrow v_2$  이다.

♡ 공감



sihyes

24학번 컴퓨터공학과



sihyes

내용을 입력하세요.

